FADENA
FACULTAD DE DEFENSA NACIONAL
Universidad de la Defensa Nacional

## Facultad de Defensa Nacional

## Licenciatura en Ciberdefensa

# Sistemas de tratamiento de Datos

## SQL Básico

**Año:** 2025

FADENA
FACULTAD DE DEFENSA NACIONAL
Universidad de la Defensa Nacional

# Introducción

## SQL

SQL (Structured Query Language) es un lenguaje estándar de definición, manipulación y control de bases de datos relacionales. Se asemeja al lenguaje natural, concretamente al inglés, ofreciendo mucha expresividad. Por este motivo, la mayor parte de los SGBD (Sistemas de Gestión de Bases de Datos) relacionales implementan este lenguaje, mediante el que realizan todo tipo de accesos a la base de datos.

Entre los SGBD que lo usan, podemos mencionar Oracle, Microsoft SQL Server, DB2 de IBM, MySQL, MariaDB, PostgreSQL, SQLite, entre otros.

Es capaz de conjugar las operaciones del álgebra y el cálculo relacional con operadores adicionales, y así definir consultas para recuperar o modificar información de bases de datos. Además de consultas, con SQL, es posible definir la estructura de los datos, modificarla, y especificar restricciones.

Teniendo en cuenta las definiciones teóricas del modelo relacional, si bien los conceptos son los mismos, para **SQL** nos referimos a **tablas** en lugar de relaciones, **columnas (o campos)** en lugar de atributos, y **filas** en lugar de tuplas. De todas formas, las consideraciones se mantienen: las tablas tienen **clave primaria**; las relaciones entre tablas se establecen mediante **claves foráneas**; los campos tienen **dominios**

definidos para cada uno; y se deben cumplir las **reglas de integridad** (de entidad, referencial, de dominio, nulos, etc.).

## TIPOS DE COMANDOS O SENTENCIAS

Los comandos o sentencias de SQL se pueden agrupar en tres categorías o conjuntos:

- **DDL** (Data Definition Language) o sentencias de definición de datos: permiten definir el esquema de bases de datos, crear tablas (con sus campos), alterar su definición y eliminarlas. Existen otros objetos además de las tablas, como las vistas, los índices y los disparadores (triggers) para los que también se usan estas sentencias, pero no serán contemplados en este momento.
- **DML** (Data Maniplation Language) o sentencias de manejo de datos: permiten insertar datos en las tablas, consultarlos, modificarlos y borrarlos. En algunas publicaciones las sentencias de consulta se referencian en un grupo separado, como DQL (Data Query Language), pero a fines prácticos es indistinto.
- **DCL** (Data Control Language) o sentencias de control:
  permiten conceder o revocar privilegios de acceso a usuarios.
  Además, hay otras opciones para resolver problemas de concurrencia en las transacciones para garantizar la integridad de los datos. No serán contemplados en este momento.

En general, las sentencias SQL se pueden escribir tanto en mayúsculas como en minúsculas, y lo mismo sucede con los nombres de las tablas y

las columnas. Para facilitar la lectura, las sentencias suelen aparecer en mayúsculas, y se agregan saltos de línea y espacios en blanco (tabulaciones) para ordenarlas. Estas sentencias terminan con el carácter punto y coma (;) para indicar donde terminan.

Para el resto del documento, consideremos que en las sintaxis propuestas lo que esté entre corchetes [...] son elementos opcionales; y las barras verticales {...|...} son separaciones entre opciones.

## SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

Para poder trabajar con bases de datos relacionales, lo primero que tenemos que hacer es definirlas. Las sentencias DDL permiten crear y borrar una base (o esquema); y crear, borrar y modificar las diferentes tablas que la componen.

En general, se usará la sentencia **CREATE** para la creación, **ALTER** para la modificación, y **DROP** para el borrado o eliminación.

### CREACIÓN DE BASES DE DATOS Y ESQUEMAS

El estándar SQL no dispone de una sentencia para la creación de base de datos como tal. En su lugar, dispone de una sentencia de creación de esquemas: **CREATE SCHEMA**. No obstante, la mayoría de los SGBD incorporan sentencias de creación de bases de datos como **CREATE DATABASE**.

Conceptualmente, podemos entender que la BD contendrá tanto la definición de los datos como los registros (datos reales), logs, etc. El esquema establecerá la definición de la base de datos, plasmando el diseño lógico en el SGBD concreto especificando las tablas y otros objetos relacionados.

En ocasiones (y dependiendo del SGBD), una base de datos puede contener múltiples esquemas para agrupar y organizar los objetos contenidos en cada uno, por ejemplo, por temática de la realidad para la que se plantea una solución.

Para algunos SGBD (como MySQL), el término "esquema" y "base de datos" se utilizan de forma indistinta porque no admite el concepto de esquema por separado.

La sintaxis simplificada para la creación de esquemas es:

- **CREATE SCHEMA** nombre_esquema;

Además de creación, dentro de DDL tenemos instrucciones para eliminación de esquemas; la sintaxis es:

- **DROP SCHEMA** nombre_esquema;

En el caso de los DBMS que lo permitan, las sentencias serán: **CREATE DATABASE** nombre_base; y **DROP DATABASE** nombre_base; respectivamente.

FADENA
FACULTAD DE DEFENSA NACIONAL
Universidad de la Defensa Nacional

## CREACIÓN DE TABLAS

Para crear una tabla se utiliza la sentencia **CREATE TABLE**. Su sintaxis es la siguiente:

**CREATE TABLE nombre_tabla**

**( definición_columna**

**[, definición_columna...]**

**[, restricciones_tabla] );**

Donde **definición_columna** es:

**nombre_columna tipo_datos [DEFAULT expr] [restricciones_columna]**

Entonces, para crear una tabla tenemos que:

1. Definir el **nombre de la tabla**: nombre_tabla.
2. Para cada **columna** de la tabla:
   - Definir el **nombre de la columna**: nombre_columna.
   - Indicar el **tipo de datos** que tendrá (su dominio): tipo_datos.
   - Opcionalmente se puede asignar un **valor por defecto**, que se utilizará cuando en una inserción no se especifique valor para esa columna: **[DEFAULT expr]**, donde expr es el valor por defecto, que puede ser NULL si el campo lo permite.
   - Opcionalmente se pueden indicar **restricciones para la columna**: [restricciones_columna].
3. Opcionalmente se pueden indicar **restricciones para la tabla**: [restricciones_tabla].

A continuación, se especifica el significado de algunas referencias que requieren mayor explicación:

- **Tipo de datos:** Los tipos de datos disponibles abarcan varios tipos numéricos, de texto, etc. También se pueden encontrar tipos definidos por el usuario. Los tipos de datos disponibles (y como se nombran) varían según el SGDB que se esté utilizando, pero por lo general presentan:
  - **CHAR(n):** Cadenas longitud fija hasta n caracteres.
  - **VARCHAR(n):** Cadenas longitud variable hasta n caracteres.
  - **INTEGER, BIGINT, SMALLINT:** Números enteros.
  - **NUMERIC(n, m), DECIMAL(n, m), REAL, FLOAT:** Números con coma.
  - **DATE, TIME, DATETIME:** Fechas, horas o ambas.
  - **BIT, BLOB**, y muchos más (varían según el SGBD).

- **Restricciones de columna:** En cada una de las columnas de la tabla, se puede imponer ciertas restricciones que se tendrán que cumplir. Pueden ser:
  - **NOT NULL**: La columna no puede tener valores nulos.
  - **NULL**: La columna admite nulos (se toma por defecto si no se especifica NOT NULL).
  - **UNIQUE**: La columna no puede tener valores repetidos. Es una clave alternativa.
  - **PRIMARY KEY**: La columna no puede tener valores repetidos ni nulos. Es la clave primaria.

- **REFERENCES** tabla [(columna)]: La columna es la clave foránea de la columna de la tabla especificada.
- **CHECK** (condiciones): La columna debe cumplir las condiciones especificadas.

- **Restricciones de tabla:** Se pueden imponer restricciones sobre toda la tabla, que se tendrán que cumplir. En general se establecen cuando hay claves compuestas (más de un campo). Pueden ser:
  - **UNIQUE** (columna [, columna...]): El conjunto de las columnas especificadas no puede tener valores repetidos. Es una clave alternativa.
  - **PRIMARY KEY** (columna [, columna...]): El conjunto de las columnas especificadas no puede tener valores nulos ni repetidos. Es una clave primaria.
  - **FOREIGN KEY** (columna [, columna...]) **REFERENCES** tabla [(columna2 [, columna2...])]: El conjunto de las columnas especificadas es una clave foránea que referencia la clave primaria formada por el conjunto de las columnas2 de la tabla dada.
  - **CHECK** (condiciones): La tabla debe cumplir las condiciones especificadas.

**Sobre las restricciones de columna y tabla:**

- **Nombre de restricción:** Se puede anteponer **CONSTRAINT nombre_restricción**, para nombrar la restricción. Si se omite, el sistema generará un nombre automáticamente.

- **Claves foráneas:** Si se definió una restricción de clave foránea, cuando se agrega o actualiza un valor en esta columna, se comprueba que dicho valor existe en la tabla referenciada. Además, **se pueden establecer reglas de comportamiento** para los casos en que se borra o actualiza el valor referenciado. La sintaxis completa para esto es:

  **FOREIGN KEY** clave_secundaria **REFERENCES** tabla [(clave_primaria)]
  **[ON DELETE {NO ACTION | CASCADE | SET DEFAULT | SET NULL}]**
  **[ON UPDATE {NO ACTION | CASCADE | SET DEFAULT | SET NULL}]**

  Donde:

  - **NO ACTION** (o RESTRICT): corresponde a la política de restricción (produce un error por intento de violación de restricción).
  - **CASCADE**: borra/actualiza las filas que hacen referencia al valor borrado/actualizado.
  - **SET NULL**: pone un nulo en las filas donde se hacía referencia al valor borrado/actualizado.
  - **SET DEFAULT**: pone el valor por defecto en las filas donde se hacía referencia al valor borrado/actualizado.

(*) A nivel de columna, se omite la parte de "FOREIGN KEY clave_secundaria".

**Ejemplo de creación de tabla:**

El siguiente ejemplo muestra la mayoría de las posibilidades vistas. No siempre será necesario establecer todo, muchas veces alcanzará con indicar los elementos básicos.

```
CREATE TABLE Productos (
        codigo_producto INTEGER,
        nombre_producto VARCHAR(50) UNIQUE NOT NULL,
        tipo VARCHAR(20),
        descripcion VARCHAR(255),
        precio REAL DEFAULT 10.0,
        fabrica INTEGER DEFAULT NULL,
        PRIMARY KEY (codigo_producto),
        FOREIGN KEY (fabrica) REFERENCES Fabricas(id_fabrica)
                        ON DELETE RESTRICT ON UPDATE CASCADE,
        CONSTRAINT precio CHECK (precio>5.0)
);
```

(*) Ver que, aunque la clave primaria no sea compuesta, se puede definir como restricción de tabla. Al definir como PRIMARY KEY, el sistema ya la asumirá como UNIQUE y NOT NULL.

## MODIFICACIÓN Y BORRADO (ELIMINACIÓN) DE TABLAS

Una vez creada una tabla, puede ser necesario agregar o eliminar una columna particular, establecer una nueva restricción, etc.

Para **modificar una tabla** se utiliza la sentencia **ALTER**. La sintaxis exacta dependerá del SGBD, pero generalmente tiene la forma:

**ALTER TABLE** nombre_tabla **ADD** nombre_columna definición_columna;

**ALTER TABLE** nombre_tabla **DROP COLUMN** nombre_columna;

**ALTER TABLE** nombre_tabla **ALTER COLUMN** nombre_columna definición_columna;

**ALTER TABLE** nombre_tabla **ADD CONSTRAINT** restricción_tabla;

**ALTER TABLE** nombre_tabla **DROP CONSTRAINT** nombre_restricción;

Las tres primeras corresponden a agregar (ADD), borrar (DROP) o modificar (ALTER o MODIFY) una **columna**; y las dos segundas a agregar (ADD) o eliminar (DROP) **restricciones** de tabla. Nuevamente, las posibilidades y sintaxis dependerán del motor de base de datos usado.

**Ejemplos de modificación de tabla:**

```
ALTER TABLE productos ADD marca VARCHAR(50);

ALTER TABLE productos DROP COLUMN fabrica;

ALTER TABLE productos
        ADD CONSTRAINT FK_Tipo FOREIGN KEY (tipo) REFERENCES tipos(id_tipo);
```

Para **eliminar una tabla** se utiliza la sentencia **DROP**, la sintaxis es:

**DROP TABLE** nombre_tabla;

(*) Ver que DROP borra o elimina la tabla, no sólo su contenido.

**En conclusión,** las sentencias DDL de SQL nos permiten trabajar con todas las definiciones vistas en el modelo relacional; creando tablas con

sus correspondientes campos, definiendo claves de diferentes tipos, relaciones entre tablas, y estableciendo reglas de integridad: de dominio (tipo de dato), de entidad, integridad referencial, nulos, etc.

## SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

Una vez creada la base de datos con sus tablas, se puede insertar, modificar y borrar los valores de las filas de las tablas. Para poder hacer esto, SQL ofrece las siguientes sentencias:

- **INSERT**: para insertar registros (filas) en la base de datos.
- **UPDATE**: para modificar los valores de los campos indicados, en los registros que cumplan cierta condición.
- **DELETE**: para eliminar los registros (filas) de una tabla que cumplan una condición.

Una vez que se han insertado valores en las tablas, estos pueden ser consultados. La sentencia para hacer consultas a una base de datos es:

- **SELECT**: permite consultar registros de la base de datos que satisfagan una condición determinada.

### INSERCIÓN DE DATOS

Se pueden **insertar valores** en una tabla de grado n, mediante la sentencia **INSERT INTO**, de dos formas:

1. Sin especificar los nombres de las columnas, sólo los valores:

   **INSERT INTO** nombre_tabla

   **VALUES** (v1, v2, ..., vn);

   Donde los valores v1, v2, ..., vn se deben corresponder exactamente (en el mismo orden) con las columnas de la tabla donde se están insertando.

2. Especificando el nombre de los $k \leq n$ atributos para los que insertamos valores:

   **INSERT INTO** nombre_tabla (col1, col2, ..., colk)

   **VALUES** (v1, v2, ..., vk);

   En este caso, los valores no indicados explícitamente toman el valor NULL, o bien el valor por defecto si se definió mediante DEFAULT.

En ambos casos deben respetarse las restricciones establecidas (de entidad, referencial, dominio, etc.).

Algunos SGBD permiten insertar varias filas en una misma tabla mediante una sola sentencia INSERT, con listas de valores separadas por coma. De esta manera puede ser más directo que si se hace mediante varias sentencias independientes.

**Ejemplos de inserción de datos:**

INSERT INTO Productos

VALUES (1001, 'MatrixQ', 'teclado', 'Teclado retroiluminado inalámbrico', 6.75, NULL);

INSERT INTO Productos

VALUES (1002, 'SuperM', 'monitor', 'Monitor LCD 24 Pulgadas', 12.15, NULL);

O bien,

INSERT INTO Productos (codigo_producto, nombre_producto, tipo, precio)

VALUES (1003, 'Zitrix', 'mouse', 6.75);

## ACTUALIZACIÓN DE DATOS

Se pueden **modificar valores** en una tabla mediante la sentencia **UPDATE SET WHERE**. La sintaxis es:

**UPDATE** Nombre_tabla

**SET** col1 = v1, col2 = v2, ..., coln = vn

**WHERE** condiciones;

Esto actualizará a los valores v1, ..., vn, las columnas col1, ..., coln, respectivamente, para todos los registros que cumplan las condiciones dadas.

**Ejemplo de actualización de datos:**

UPDATE Productos

SET precio = precio * 1.10

WHERE tipo = 'monitor';

## ELIMINACIÓN DE REGISTROS

Se pueden **eliminar registros (filas)** de una tabla mediante la sentencia **DELETE FROM WHERE**. La sintaxis es:

```
DELETE FROM Nombre_tabla
WHERE condiciones;
```

Esto eliminará los registros de la tabla que cumplan las condiciones dadas.

**Ejemplo de eliminar registros:**

```
DELETE FROM Productos
WHERE codigo_producto = 1005;
```

**Sobre las actualizaciones y borrados:**

Si se quisiera actualizar campos para todas las filas de una tabla; o borrar todas las filas de una tabla, se usaría sólo las sentencias UPDATE o DELETE FROM, **sin el WHERE**. Si no es la intención, hay que prestar especial atención en evitarlo, porque podría llevar a la pérdida de datos o modificaciones erróneas en toda la tabla.

**Para** comprender completamente el funcionamiento de **UPDATE** y **DELETE es necesario conocer bien el funcionamiento de la sentencia SELECT**. Esto es así porque para poder actualizar o eliminar datos que se han almacenado es preciso encontrarlos antes. Por lo tanto, para estas sentencias se establecen las condiciones de búsqueda de dichos datos

(WHERE) del mismo modo que las condiciones de búsqueda cuando se hace una consulta.

## CONSULTA DE DATOS

Se pueden hacer **consultas** sobre una tabla mediante la sentencia **SELECT FROM**. Consideremos la siguiente sintaxis básica:

**SELECT** [ **DISTINCT** ] { * | columna [**AS** columna_renombrada] [ , columna… ] }
**FROM** tabla [**AS** tabla_renombrada]
[ **WHERE** condición_de_búsqueda ]
[ **ORDER BY** columna **[ ASC | DESC ]** [,columna [ ASC | DESC ] ];

- La opción **AS**: permite (si se quisiera) renombrar las columnas o tablas que queremos consultar (permite definir alias). Es habitual poner sólo un espacio en blanco entre el nombre original y el alias (omitiendo la palabra AS).

El orden en que se tienen en cuenta las distintas cláusulas durante la ejecución, y la función de cada una de ellas, es la siguiente:

- **FROM**: especifica la tabla sobre la que se va a realizar la consulta.
- **WHERE**: si sólo se debe mostrar un subconjunto de las filas de la tabla, aquí se especifica la condición que deben cumplir las filas a mostrar; esta condición será un predicado booleano con comparaciones unidas por AND y OR. Si no se especifica, devolverá todas las filas de la tabla.

- **SELECT**: aquí se especifican las columnas a mostrar en el resultado; para mostrar todas las columnas se utiliza * (asterisco).
- **DISTINCT**: es una palabra clave que se puede utilizar tras la cláusula SELECT para que no se muestren filas repetidas en el resultado (esto puede ocurrir sólo cuando en la cláusula SELECT se prescinde de la clave primaria de la tabla).
- **ORDER BY**: se puede utilizar para ordenar el resultado de la consulta. Si se incluye, es siempre la última en la sentencia. La ordenación puede ser ascendente (por defecto) o descendente, y se puede basar en una o varias columnas.

**(*)** Algunos DBMS tienen la opción de **limitar el número de filas a recuperar**. Por ejemplo, MariaDB, PostgreSQL, etc. usan la cláusula **LIMIT número**, al final de la consulta; SQL Server usa **TOP número** después del SELECT, etc.

**Operadores para la cláusula WHERE**

Para definir las condiciones de la cláusula WHERE, se pueden usar los operadores de los que dispone SQL:

- **Operadores lógicos:** permiten formar condiciones más complejas a partir de otras pasadas como operandos. Así, dadas tres condiciones (expresiones lógicas) a, b y c:
  - **a AND b**: conjunción, evalúa las condiciones a y b y devuelve verdad si ambas son ciertas.

- **a OR b**: disyunción lógica, evalúa a y b y devuelve verdad si alguna de las dos es cierta.
- **NOT c**: negación lógica, devuelve el valor lógico contrario a la evaluación de c.

(*) En SQL, el orden de los operadores lógicos, de mayor a menor precedencia, es NOT, luego AND y finalmente OR. Los operadores de comparación, aritméticos y de otro tipo se evalúan antes que los lógicos, pero su orden específico varía según el tipo de operación. Para evitar ambigüedad y mejorar la legibilidad, se recomienda el uso de paréntesis para agrupar las condiciones y forzar un orden de evaluación deseado.

- **Operadores de comparación:** para cada condición simple, antes de agruparlas mediante los operadores lógicos anteriores, podemos emplear operadores como:
  - **=** Igual que, **<>** Distinto de
  - **<** Menor que, **>** Mayor que
  - **<=** Menor o igual que, **>=** Mayor o igual que

- **Otros operadores especiales:** también se pueden emplear operadores especiales como:
  - **BETWEEN: columna BETWEEN a AND b**, devuelve los registros en los que el valor de la columna esté entre los límites a y b, ambos inclusive. Por ej.: precio BETWEEN 5.0 AND 20.0.

- **IN: columna IN (valor1, …, valorN)**, devuelve los registros donde la columna indicada coincide con alguno de los dados en una lista. Por ej.: provincia IN ('Mendoza', 'Tucumán', 'Jujuy').
- **LIKE: columna LIKE característica**, permite comprobar si una columna de texto cumple ciertos patrones. En general la característica es un texto donde se usan "comodines": **_ (guión bajo) para considerar caracteres individuales, y % para una secuencia de cero o más caracteres**. Por ej., Apellido LIKE 'S%' (apellidos que empiezan con S).
- **IS NULL: columna IS NULL**, permite comprobar si un valor es nulo. Para saber si una columna no es nula, el operador es **IS NOT NULL**. Recordemos que un nulo no es un valor, implica la ausencia de valor.

**Operadores y funciones para las cláusulas SELECT y WHERE**

En las cláusulas SELECT y WHERE, además de columnas, también se pueden incluir expresiones que contengan columnas y constantes, así como funciones. Dependerán del SGBD que se utilice, en general se pueden incluir:

- **Operadores y funciones matemáticas:**
  - **Operadores entre números:** n + m (suma), n - m (resta), n * m (multiplicación), n / m (división), n % m (módulo o resto).
  - **Funciones matemáticas:** ABS(n) (valor absoluto), SQRT(n) (raíz cuadrada), POWER(n, e) (potencia de n elevado a e), etc.

- **Operadores y funciones de cadenas de caracteres:**
  - **t || r ó t + r ó CONCAT(t, r,..)**: concatena cadenas.
  - **UPPER(t) y LOWER(t)**: convierte a mayúsculas o minúsculas.
  - **LEN(t) ó LENGTH(t)**: devuelve el número de caracteres.
  - **REPLACE(cadena, a_buscar, a_reemplazar)**: reemplaza las ocurrencias de un texto por otro.
  - **SUBSTRING(cadena, inicio, longitud)**: devuelve la subcadena especificada a partir de una posición.
  - **LTRIM(t), RTRIM(t), TRIM(t)**: elimina espacios en blanco a la izquierda, derecha, o ambos.
  - Otros…

- **Operadores y funciones de fecha:**
  - **EXTRACT(parte FROM fecha) ó DATEPART(parte, fecha) ó DATE_PART(parte, fecha)**: extrae una parte de la fecha (año, mes, día).
  - **YEAR(fecha), MONTH(fecha), DAY(fecha)**: extrae año, mes o día de la fecha.
  - **DATEDIFF(fecha1, fecha2)**: calcula la diferencia entre dos fechas.
  - **DATEADD(intervalo, n, f)**: suma o resta un intervalo especificado (días, meses, años) a una fecha.
  - **CURRENT_DATE ó GETDATE()**: devuelve la fecha actual del sistema.
  - Otros…

Ejemplo de consulta de datos:

```
SELECT DISTINCT codcli, EXTRACT(month FROM fecha) AS meses
 FROM facturas
WHERE codcli IN (45, 54, 87, 102)
 ORDER BY fecha DESC;
```

**Funciones de agregación (o agregado)**

SQL ofrece una serie de funciones que permiten contar datos o hacer cálculos sobre ellos. Se usan dentro de una cláusula SELECT y actúan sobre los valores de las columnas. Los más habituales son:

- **COUNT(*)**: Cuenta filas.
- **COUNT(columna)**: Cuenta valores no nulos de la columna.
- **COUNT(DISTINCT columna)**: Cuenta valores distintos y no nulos de la columna.
- **SUM(columna)**: Suma los valores de la columna.
- **MAX(columna):** Obtiene el valor máximo de la columna.
- **MIN(columna):** Obtiene el valor mínimo de la columna.
- **AVG(columna):** Obtiene la media de los valores de la columna.

Si no se realiza ninguna restricción en la cláusula WHERE de una sentencia SELECT que utiliza estas funciones, éstas se aplican sobre todas las filas de la tabla especificada en la cláusula FROM. Sin embargo, cuando se realiza una restricción mediante WHERE, las funciones se aplican sólo sobre las filas que la restricción ha seleccionado.

Por ejemplo:

```
SELECT MAX(precio) AS pr_máx_monitores, AVG(precio) AS pr_prom_monitores
  FROM Productos
WHERE tipo = 'monitor';
```

## CONSULTAS CON AGRUPACIÓN DE FILAS

Consideremos la siguiente sintaxis de la sentencia SELECT, que amplía la original para incluir las posibilidades de agrupación:

```
SELECT columna [ , columna ]
  FROM tabla
[ WHERE condición_de_búsqueda ]
[ GROUP BY columna [, columna ]
[ HAVING condición_para_el_grupo ] ]
[ ORDER BY columna [ ASC | DESC ] [,columna [ ASC | DESC ] ];
```

**Cláusula GROUP BY (Agrupamiento):** La cláusula **GROUP BY forma grupos con las filas que tienen en común los valores (son iguales) de una o varias columnas**. Para cada grupo, se mostrará una única fila en el resultado. Sobre cada grupo se pueden aplicar las funciones de agregación que se vieron (SUM, MAX, MIN, AVG, COUNT), que pasan a denominarse funciones de grupo. Estas funciones, utilizadas en la cláusula SELECT, se aplican una vez para cada grupo.

**Cláusula HAVING:** Una vez que GROUP BY ha combinado las filas en grupos, se puede incluir a continuación la cláusula **HAVING** que usa las **funciones de grupo (cálculos agregados) para filtrar** los registros agrupados que satisfagan ciertas condiciones. Es similar al WHERE, pero a diferencia de éste (que afecta a todas las filas de la tabla), HAVING aplica condiciones sobre datos calculados para los grupos generados por GROUP BY.

En las consultas que utilizan GROUP BY, se parte de las filas de la tabla que cumplen el predicado establecido en la cláusula WHERE, y se agrupan en función de los valores comunes en la columna o columnas especificadas (se muestra una fila por cada uno de los grupos obtenidos). Mediante la cláusula HAVING se seleccionan aquellos grupos obtenidos por la cláusula GROUP BY, que cumplen el predicado establecido en la condición.

Tanto en la cláusula SELECT como en HAVING sólo pueden aparecer las columnas que se incluyen en la cláusula GROUP BY, y/o las funciones de agregación sobre otras columnas. Si en las cláusulas SELECT o HAVING aparecen columnas que no se han especificado en la cláusula GROUP BY o que no están afectadas por una función de grupo, se produce un error.

Ejemplo de consulta de agrupación:

```
SELECT tipo, AVG(precio) AS precio_promedio
  FROM Productos
 GROUP BY tipo
HAVING COUNT(*) >= 2;
```

Devolverá una fila por cada tipo de producto, mostrando el tipo y el precio promedio de los productos de ese tipo, siempre que haya al menos 2 productos de ese tipo. Si se omite el HAVING, se obtendrá una fila por cada tipo existente.

## CONSULTAS A MÁS DE UNA TABLA

Muchas veces se necesita consultar datos de más de una tabla haciendo combinaciones de columnas de tablas diferentes. Esto implica conceptos vistos en el álgebra relacional, como el producto cartesiano y la concatenación. En SQL es posible listar más de una tabla que se quiere consultar especificándolo en la cláusula FROM.

El producto cartesiano se lleva a cabo -en su forma más elemental- separando las **tablas involucradas por una coma (,),** por ejemplo:

```
SELECT *
FROM facturas, clientes;
```

La sentencia anterior combina todas las filas de la tabla facturas con todas las filas de la tabla clientes. Si la primera tiene n filas y r columnas; y la segunda tiene m filas y s columnas, el resultado tendrá n × m filas y r + s columnas.

En general, se denomina JOIN al producto cartesiano filtrado por ciertas condiciones que restrinjan el resultado a los registros las cumplan (usando valores de ciertas columnas de las tablas involucradas). Si hay

campos con el mismo nombre, los prefijamos. Por ejemplo, para que lo anterior tenga más sentido:

```
SELECT *
  FROM facturas, clientes
WHERE facturas.codcli = clientes.codcli;
```

Hay otra forma adicional, que **es más explícita y clara** a la hora de realizar este tipo de combinaciones. La sintaxis es la siguiente, usando la **cláusula JOIN (ó INNER JOIN)**:

```
SELECT [ DISTINCT ] { * | [lista_columnas] }
  FROM tabla1 JOIN tabla2 ON condiciones_vínculos_tablas
[ WHERE condiciones_de_búsqueda ]
```

De esta manera se relacionan de manera más limpia ambas tablas, porque permite identificar más claramente qué son restricciones (aparecerán en el WHERE) y qué son condiciones de concatenación (aparecerán en el FROM con las palabras clave **JOIN** y **ON**). La opción **ON** es usada para definir el criterio de selección, utiliza una expresión booleana para retornar verdadero o falso tal como en la cláusula WHERE.

Por ejemplo:

```
SELECT P.codigo_proyecto, P.precio, C.apellido
  FROM clientes C JOIN proyectos P ON C.codigo_cli = P.codigo_cliente
WHERE C.codigo_cli = 20;
```

Devolverá el código del proyecto, su precio y apellido del cliente al que corresponde, donde el código del cliente sea 20. Ver que se usaron alias para las tablas (sin la palabra clave AS, sólo con el espacio).

Este tipo de relaciones **se puede ir aplicando a cuantas tablas se necesiten** combinar en las consultas. Es decir, si se quiere combinar **tres o más tablas**, hay que **ir haciendo combinaciones de tablas por pares**. Por ejemplo:

```
SELECT P.id_pedido, C.id_cliente, C.nombreEmpresa, P.fecha
  FROM Clientes C
        JOIN Pedidos P ON C.id_cliente = P.id_cliente
        JOIN Empleados E ON P.id_empleado = E.id_empleado
WHERE C.pais = 'Argentina' OR E.id_empleado = 5;
```

También se puede utilizar una misma tabla dos veces con alias diferentes, para distinguirlas. Para hacer **JOIN a la propia tabla**, lo más común es **renombrar** dos veces la tabla. Por ejemplo:

```
SELECT A.nombre AS supervisor, B.nombre AS supervisado
  FROM Empleados A INNER JOIN Empleados B ON A.id = B.id_supevisor;
```

Además de expresar condiciones con la igualdad, la cláusula **ON** nos ofrece la **posibilidad** de expresar condiciones con los **demás operadores de comparación que no sean el de igualdad**. Sería el equivalente a la

operación que en álgebra relacional que llamábamos Theta Join. Por ejemplo:

```
SELECT E1.codigo_empl, E1.apellido_empl
  FROM empleados E1 JOIN empleados E2 ON E1.sueldo > E2.sueldo
WHERE E2.codigo_empl = 5;
```

Devolverá los códigos y apellidos de los empleados que ganan más que el empleado que tiene código 5.

Algunos SGBD permiten la combinación natural (natural join) de dos tablas, combinándolas cuando las columnas que se llaman igual tienen los mismos valores. Las columnas por las que se hace la concatenación aparecen una sola vez en el resultado. En estos casos las sintaxis son del tipo tabla1 **NATURAL JOIN** tabla2. De todas formas, por lo general **es aconsejable usar JOIN con ON**, explicitando las condiciones de vínculos entre tablas.

La combinación interna (inner join) sólo se queda con las filas que tienen valores idénticos en las columnas de las tablas que compara. Esto puede hacer que perdamos alguna fila interesante de alguna de las dos tablas; por ejemplo, porque se encuentra NULL en el momento de hacer la combinación, o porque no hay coincidencia. Por esto **SQL dispone de la combinación externa (OUTER JOIN)**, que permite obtener todos los valores de la tabla que se ha puesto a la izquierda, los de la tabla que se ha

puesto a la derecha, o todos los valores de las dos tablas (**LEFT**, **RIGHT** y **OUTER**). Su formato es:

```
SELECT columnas_a_seleccionar
  FROM T1 [LEFT|RIGHT|FULL] [OUTER] JOIN T2 ON condiciones_vínculos_tablas
[ WHERE condiciones_de_búsqueda ];
```

Supongamos el caso de **LEFT (OUTER) JOIN**: en primer lugar, se realiza un **INNER JOIN**. Entonces, para cada fila en T1 que no satisface la condición de unión con cualquier fila en T2, **se agrega una fila unida con valores nulos en columnas** de T2. Por lo tanto, **el resultado tiene incondicionalmente al menos una fila para cada fila** en T1. De manera similar lo hará para el caso de RIGHT y FULL, completando con nulos las filas para las que no haya coincidencias. Por ejemplo:

```
SELECT E.nombre_empleado, TE.telefono_número
  FROM empleados E LEFT OUTER JOIN telefonosEmp TE
                                ON E.id_empleado = TE.id_empleado;
```

Devolverá el nombre de los empleados y sus números de teléfono (una fila por cada uno); y si no tuviera teléfono, igualmente mostrará el nombre del empleado y un NULL para el teléfono.

**En conclusión,** las sentencias DML de SQL nos permiten llevar a cabo todas las operaciones vistas en el álgebra relacional; seleccionar y filtrar campos y filas, renombrar campos y tablas, usar funciones de agregación, estableciendo relaciones entre tablas, etc.